On Gated Permutations and Optimal Non-Collapsing Hash Functions

Joint work with Joseph Carolan and Vinod Vaikuntanathan

Motivation I: Random Permutations

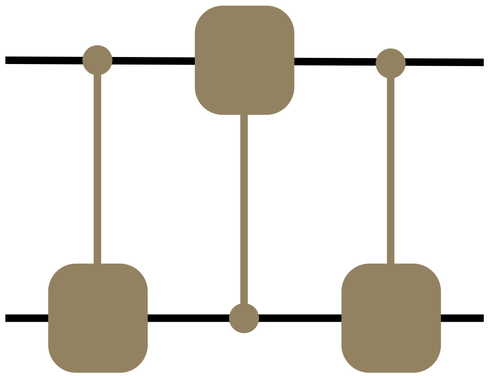
- Random permutations are used extensively in cryptography

Motivation I: Random Permutations

- Random permutations are used extensively in cryptography
- Permutations provide structure which is lacking in generic functions

Motivation I: Random Permutations

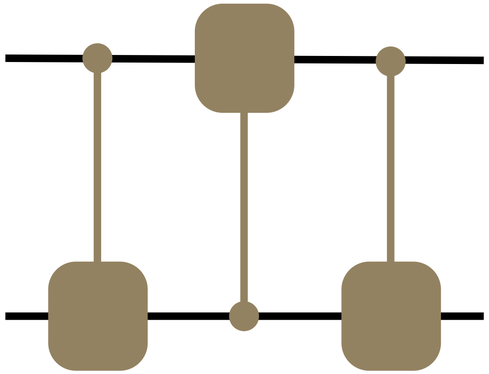
- Random permutations are used extensively in cryptography
- Permutations provide structure which is lacking in generic functions



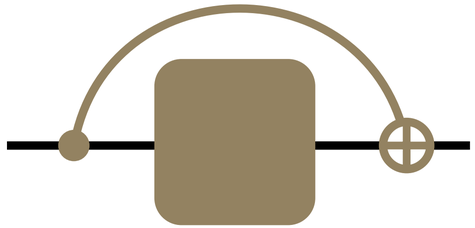
Feistel (DES)

Motivation I: Random Permutations

- Random permutations are used extensively in cryptography
- Permutations provide structure which is lacking in generic functions



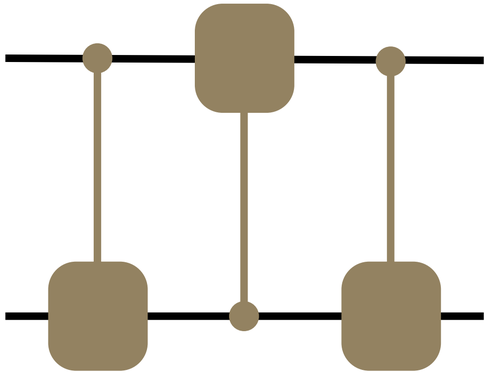
Feistel (DES)



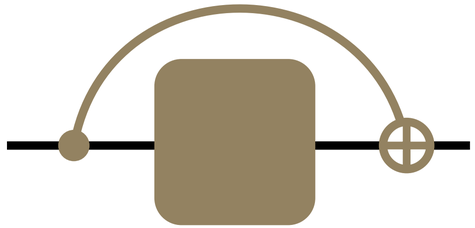
Davies Meyer
(SHA1,2)

Motivation I: Random Permutations

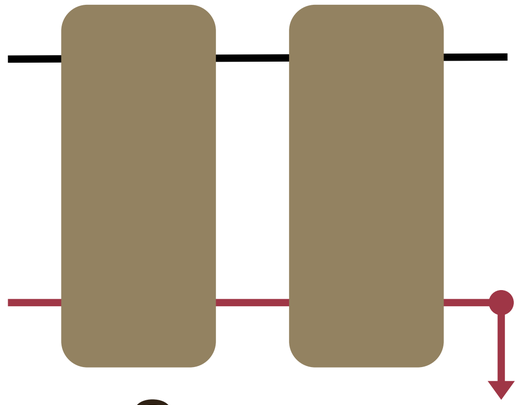
- Random permutations are used extensively in cryptography
- Permutations provide structure which is lacking in generic functions



Feistel (DES)



Davies Meyer
(SHA1,2)



Sponge
(SHA3)

Motivation I: Random Permutations

- In the *random permutation model*, all parties are given oracle access to a **random permutation** $\pi : \{0,1\}^n \to \{0,1\}^n$ as well as its **inverse** π^{-1}

Motivation I: Random Permutations

- In the *random permutation model*, all parties are given oracle access to a **random permutation** $\pi : \{0,1\}^n \to \{0,1\}^n$ as well as its **inverse** π^{-1}
- We consider giving oracle access to π^{-1} as well as π because in many constructions (e.g. key-alternating ciphers), the underlying permutation is constructed to be as easy to invert as it is to compute

Motivation I: Random Permutations

- In the *random permutation model*, all parties are given oracle access to a **random permutation** $\pi : \{0,1\}^n \to \{0,1\}^n$ as well as its **inverse** π^{-1}
- We consider giving oracle access to π^{-1} as well as π because in many constructions (e.g. key-alternating ciphers), the underlying permutation is constructed to be as easy to invert as it is to compute
- Until recently, post-quantum security has not been studied very closely

Motivation I: Random Permutations

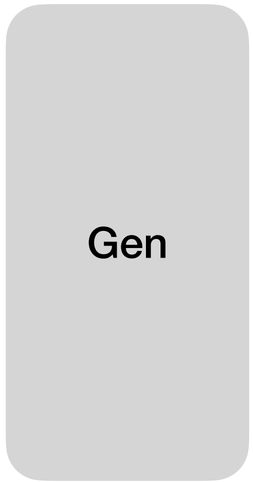
- In the *random permutation model*, all parties are given oracle access to a **random permutation** $\pi : \{0,1\}^n \to \{0,1\}^n$ as well as its **inverse** π^{-1}
- We consider giving oracle access to π^{-1} as well as π because in many constructions (e.g. key-alternating ciphers), the underlying permutation is constructed to be as easy to invert as it is to compute
- Until recently, post-quantum security has not been studied very closely
- The work of [Car25], building on [ACMT25], shows how to analyze random permutations via the *compressed permutation oracle*

Motivation II: Unclonable Cryptography

- Digital signature scheme:

Motivation II: Unclonable Cryptography

- Digital signature scheme:

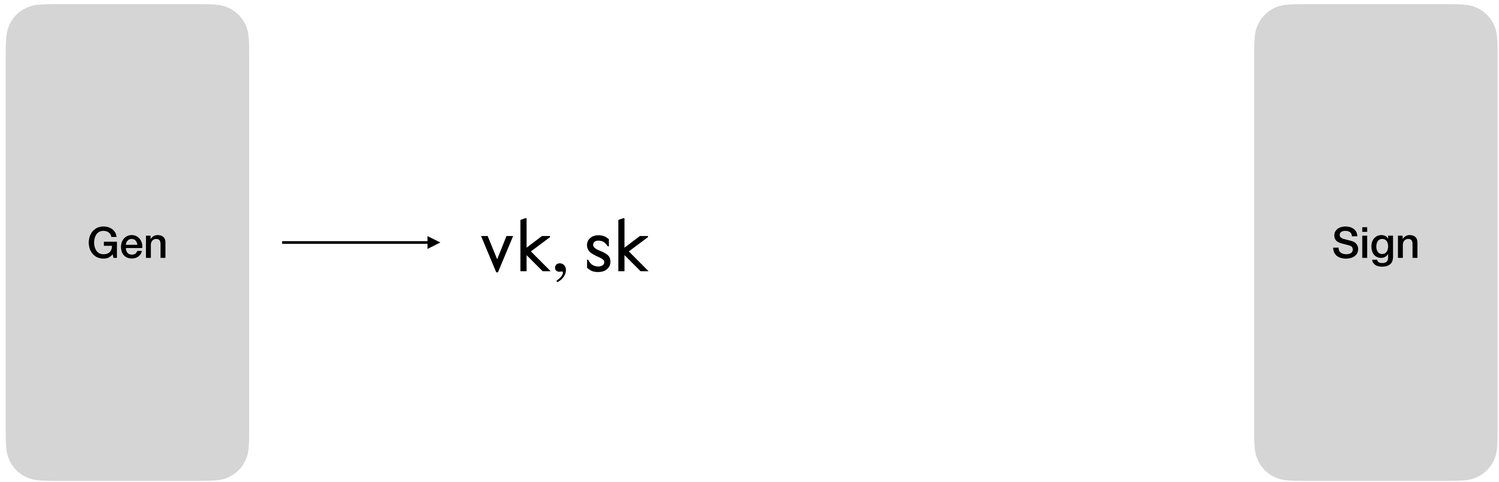


Motivation II: Unclonable Cryptography

- Digital signature scheme:

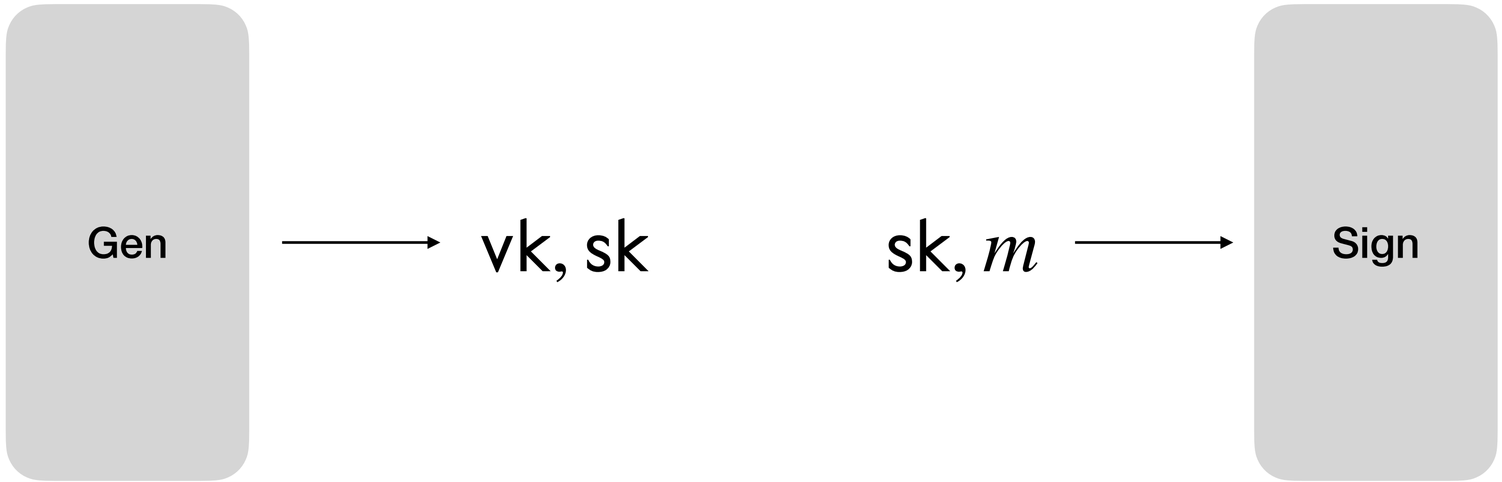
Motivation II: Unclonable Cryptography

- Digital signature scheme:



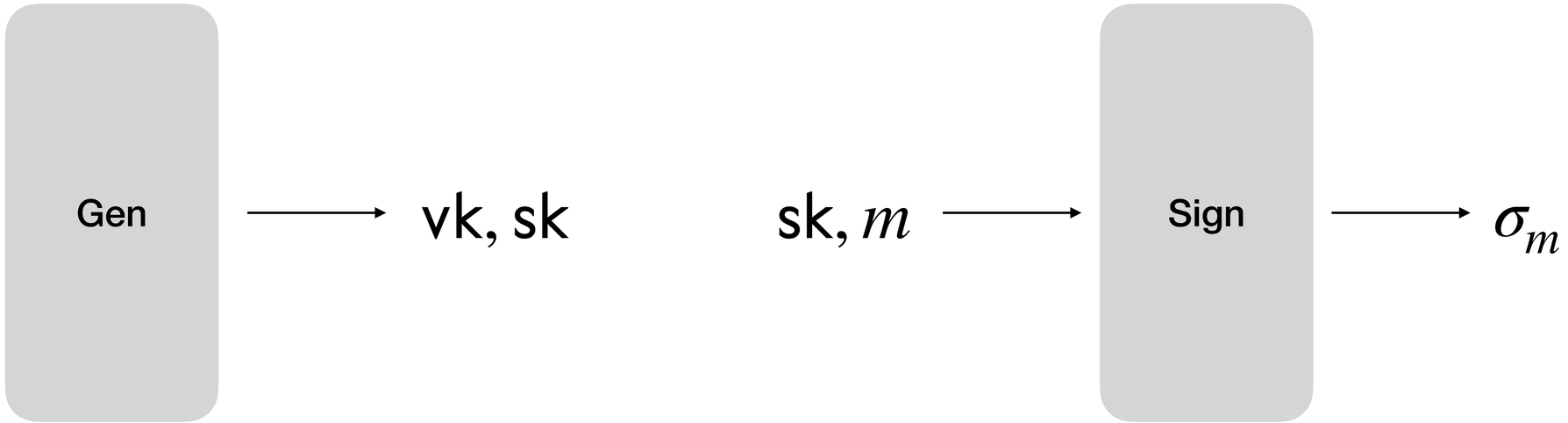
Motivation II: Unclonable Cryptography

- Digital signature scheme:



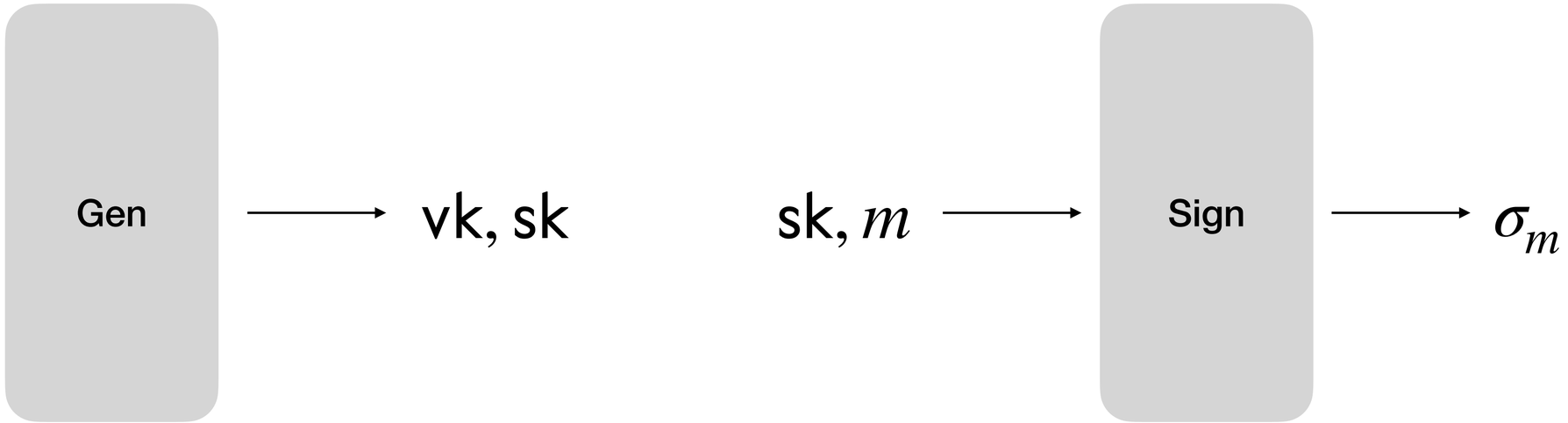
Motivation II: Unclonable Cryptography

- Digital signature scheme:



Motivation II: Unclonable Cryptography

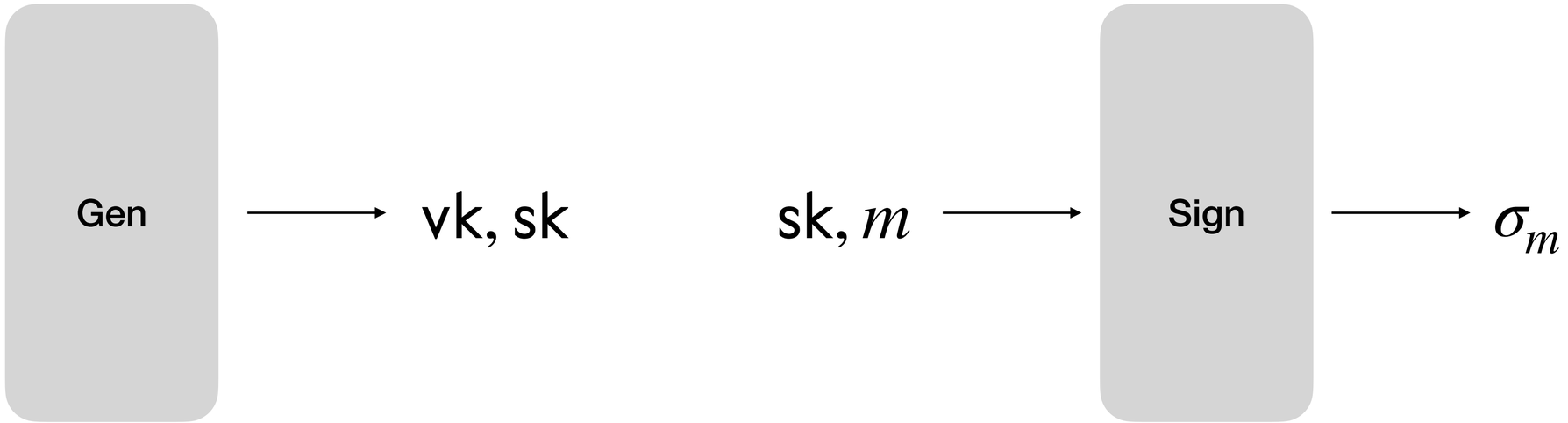
- Digital signature scheme:



- Verification is **public**: **anyone** with the verification key can verify

Motivation II: Unclonable Cryptography

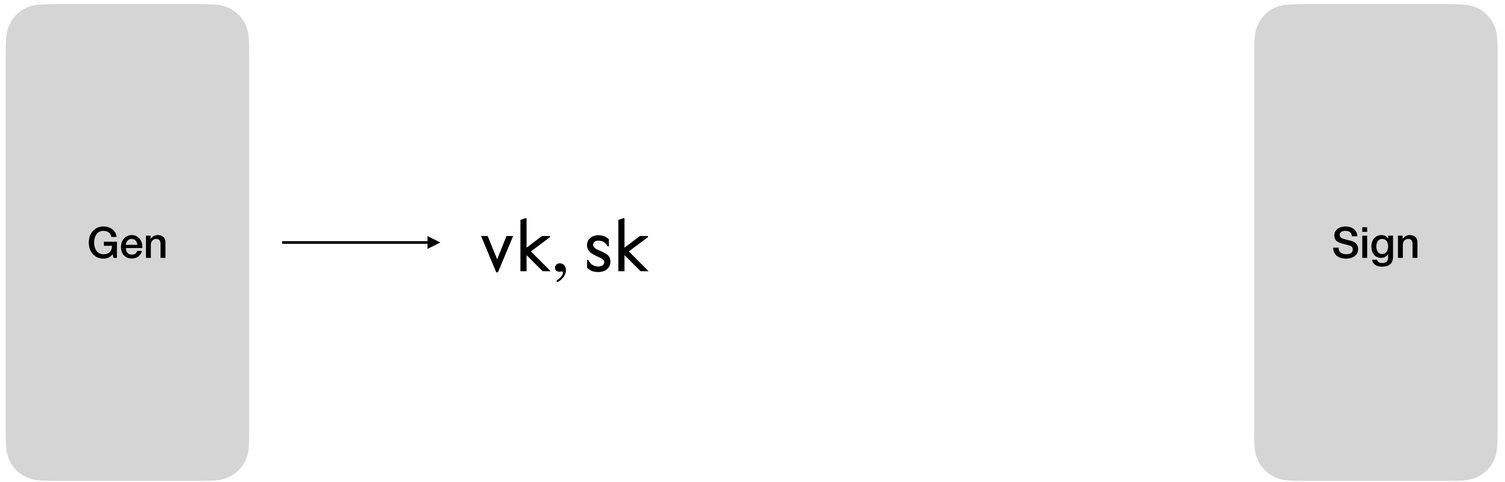
- Digital signature scheme:



- Verification is **public**: **anyone** with the verification key can verify
- **Security**: *only the owner of the signing key* can produce valid signatures

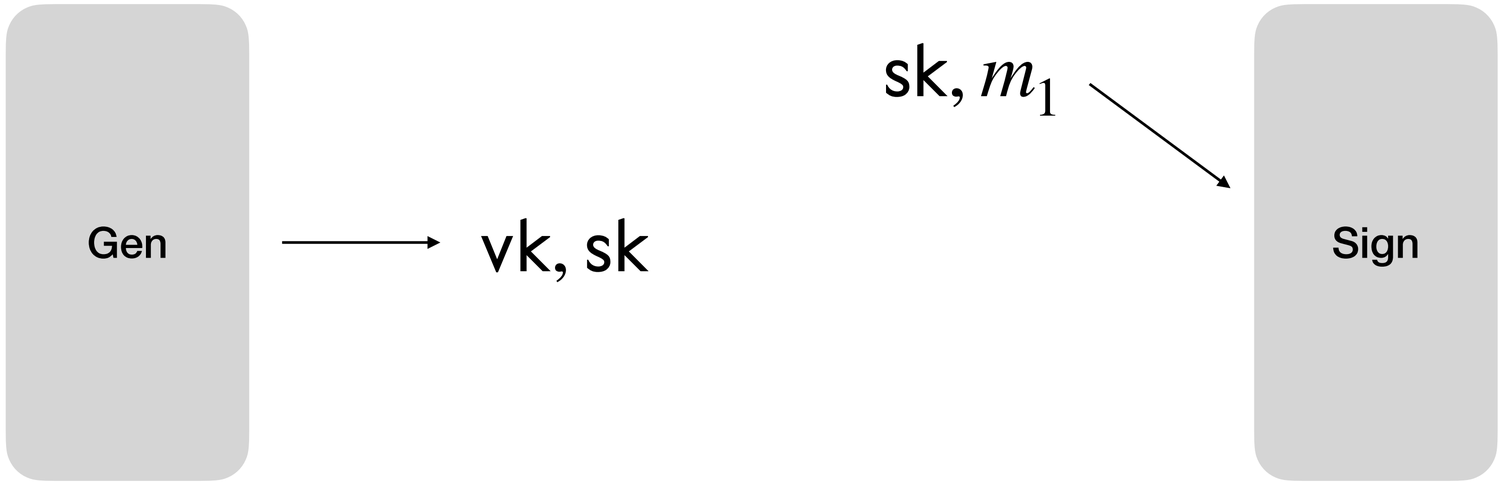
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



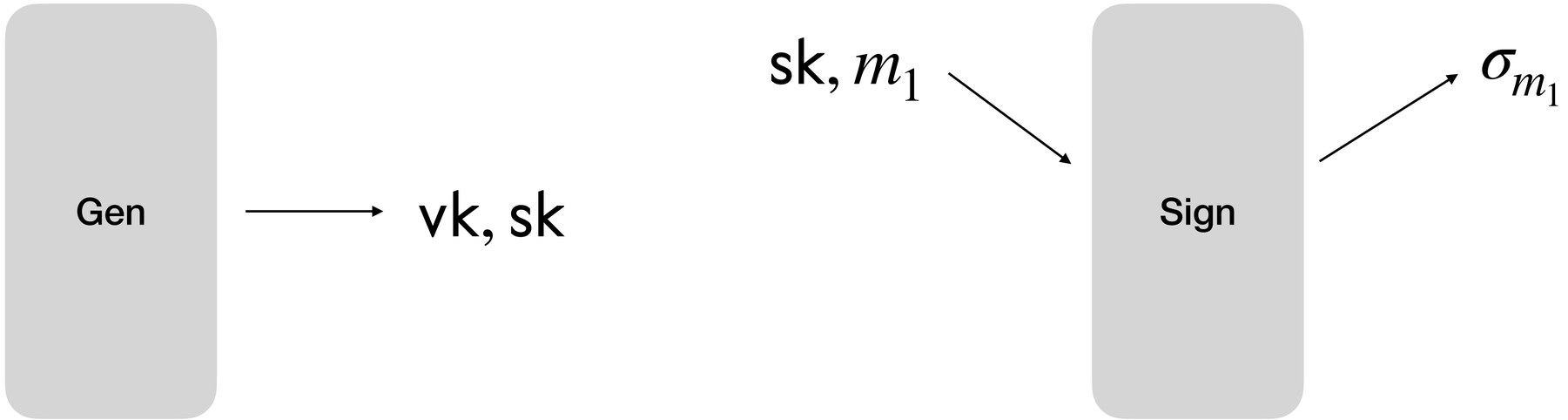
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



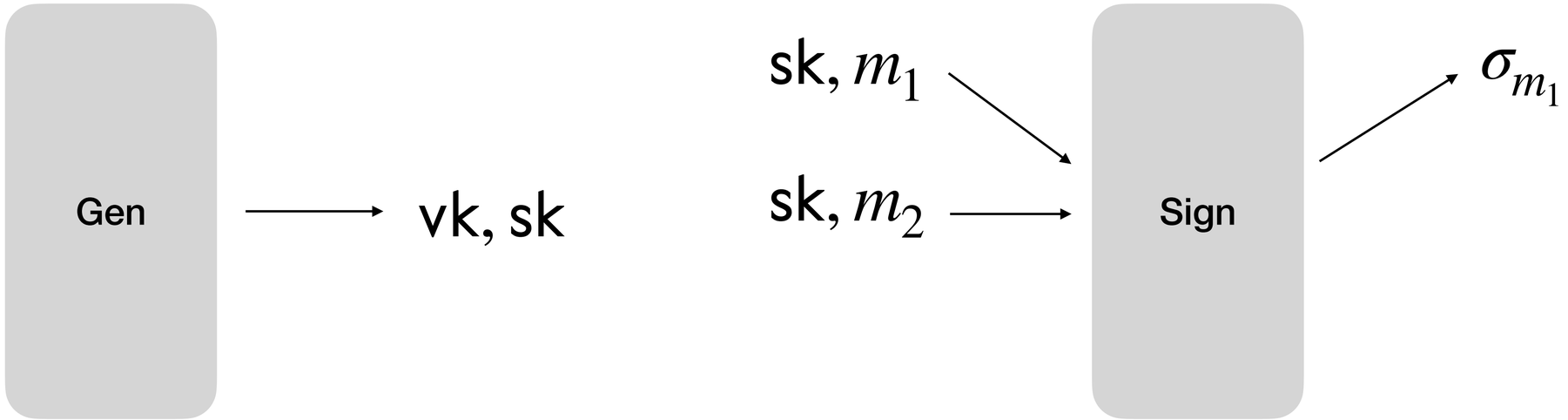
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



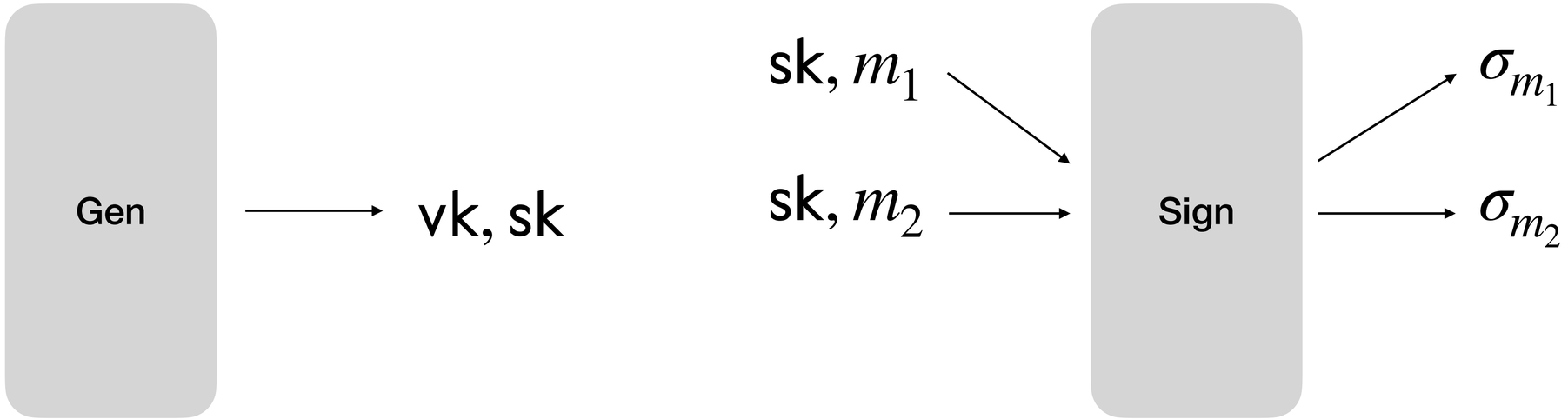
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



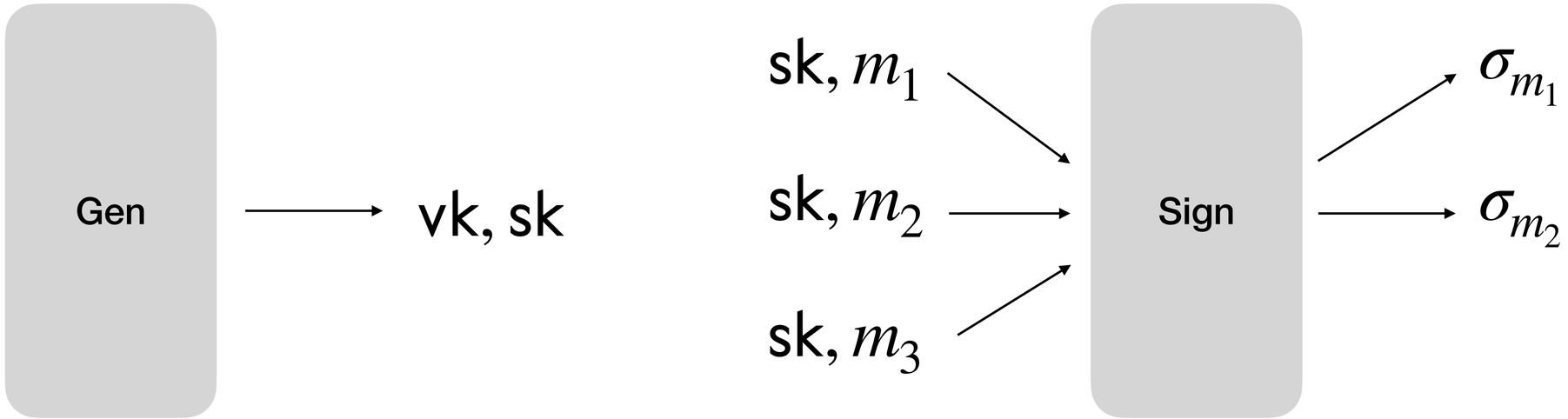
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



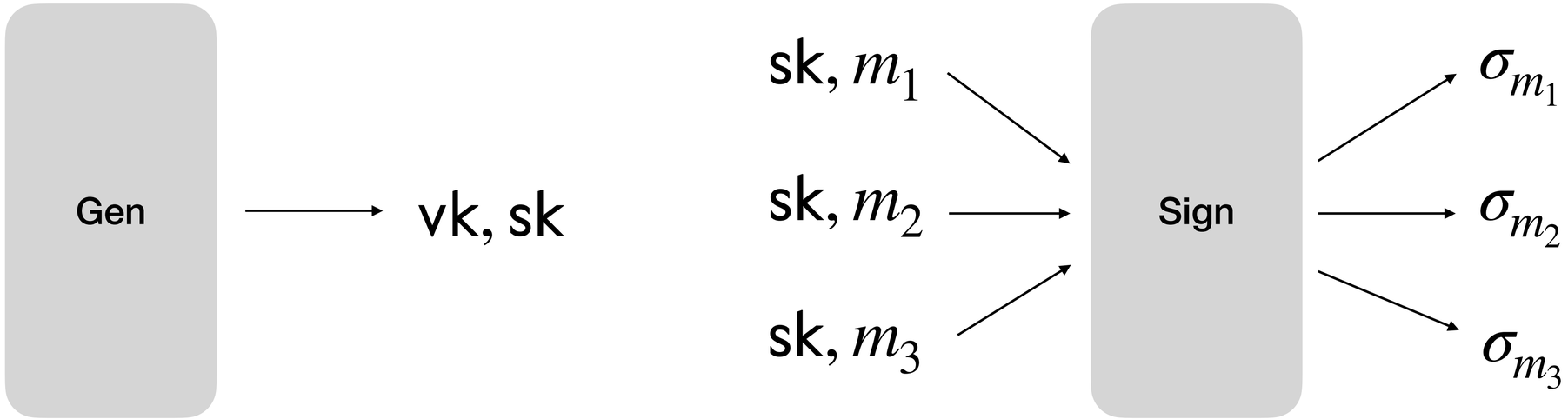
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



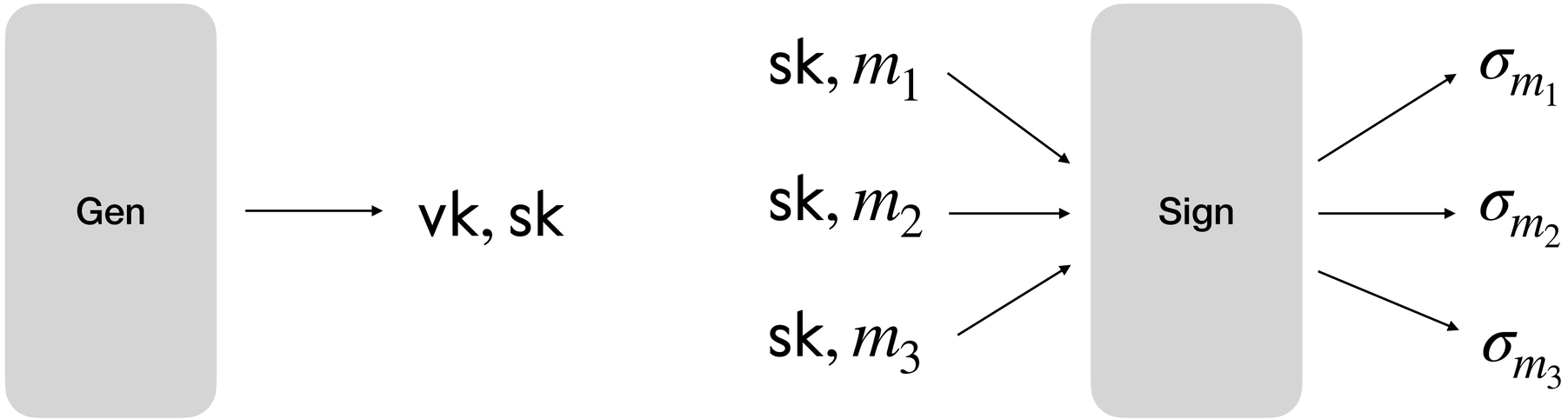
Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



Motivation II: Unclonable Cryptography

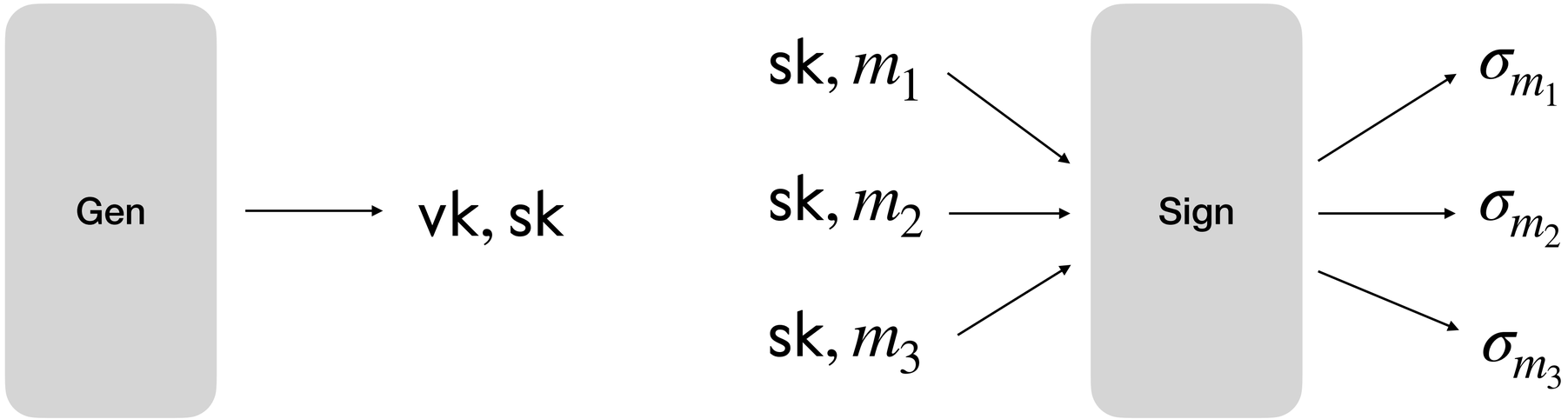
- What if Alice wants to allow Bob to sign a message on her behalf?



- Q: Is it possible to construct a *one-time* signature scheme?

Motivation II: Unclonable Cryptography

- What if Alice wants to allow Bob to sign a message on her behalf?



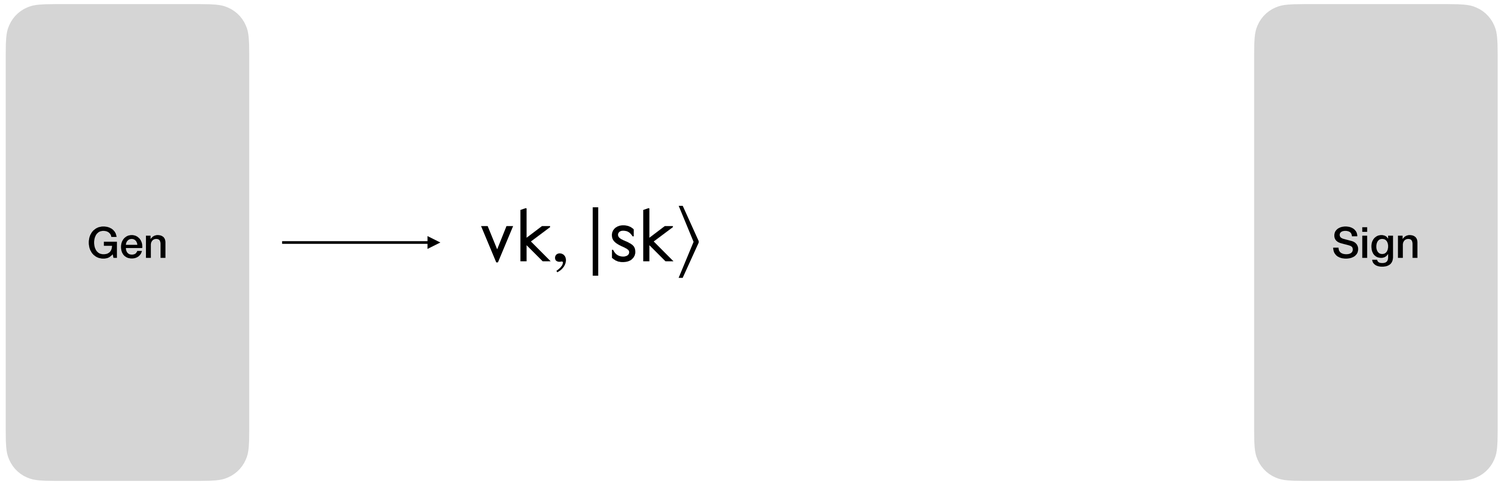
- Q: Is it possible to construct a *one-time* signature scheme?
- Clearly impossible in the classical model…

Motivation II: Unclonable Cryptography

- One-shot signatures [AGKZ20, SZ25]:

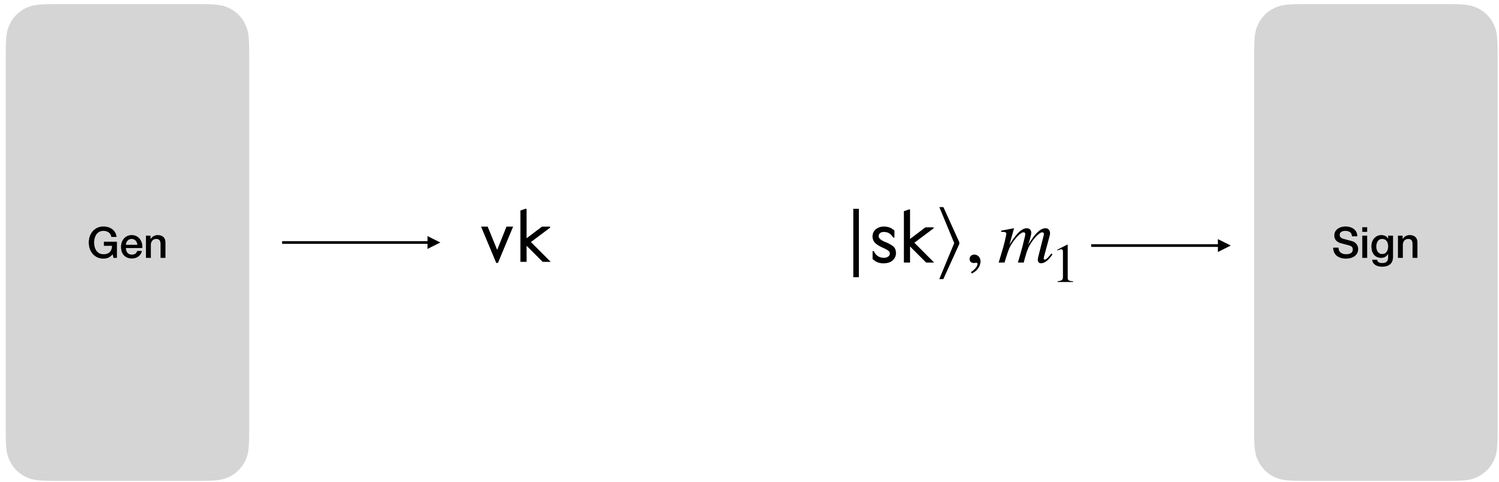
Motivation II: Unclonable Cryptography

- One-shot signatures [AGKZ20, SZ25]:



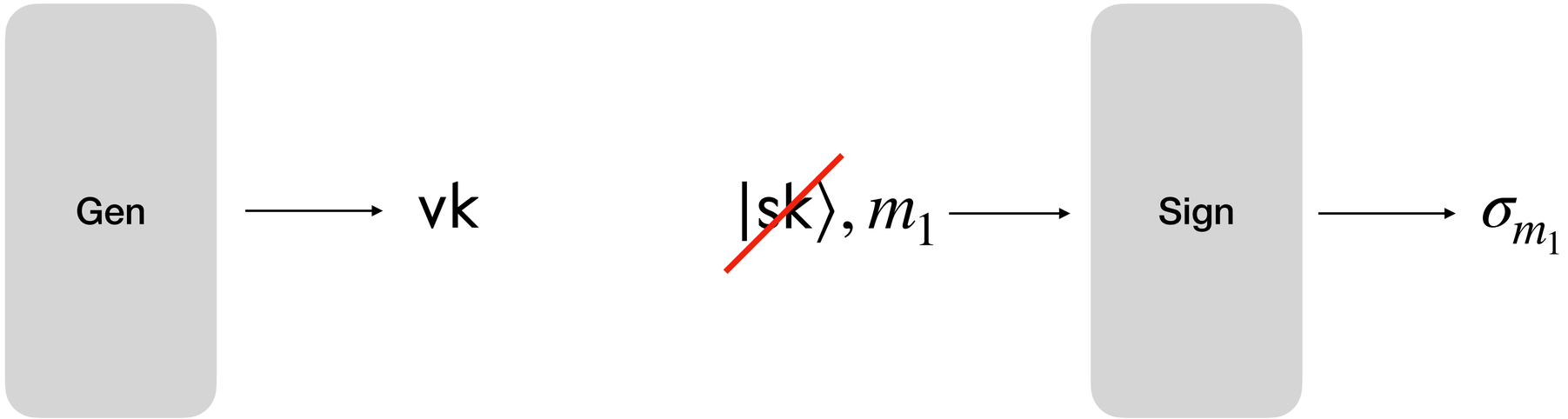
Motivation II: Unclonable Cryptography

- One-shot signatures [AGKZ20, SZ25]:



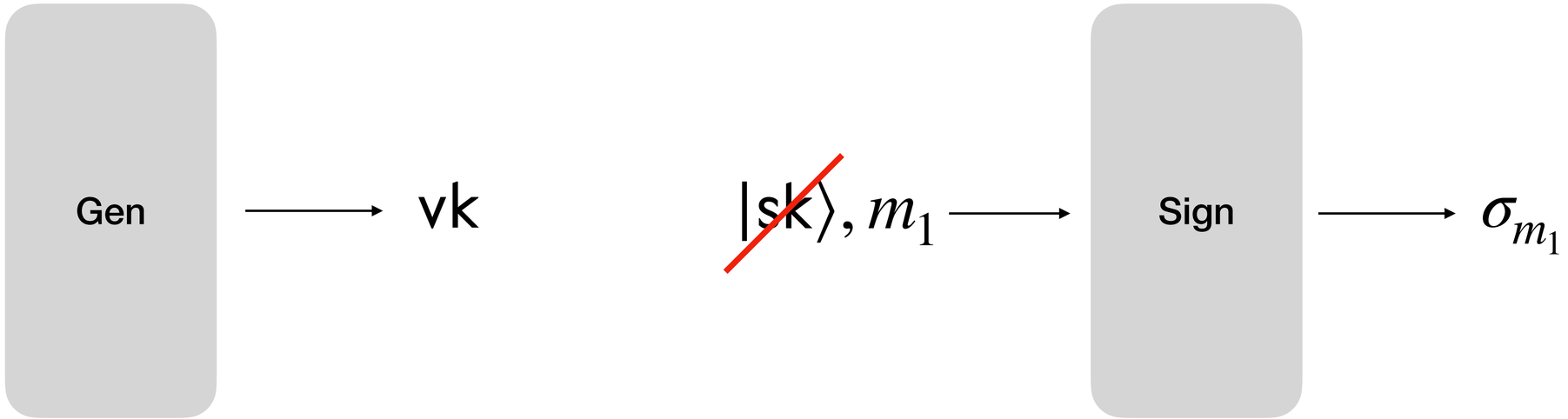
Motivation II: Unclonable Cryptography

- One-shot signatures [AGKZ20, SZ25]:



Motivation II: Unclonable Cryptography

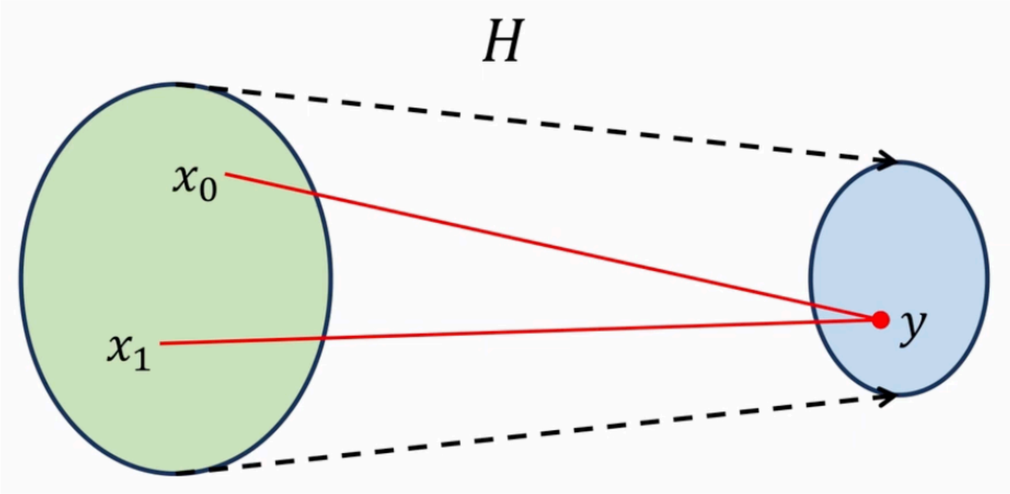
- One-shot signatures [AGKZ20, SZ25]:



- Implies quantum lightning, quantum money with classical communication, proofs of quantumness, certified randomness, …

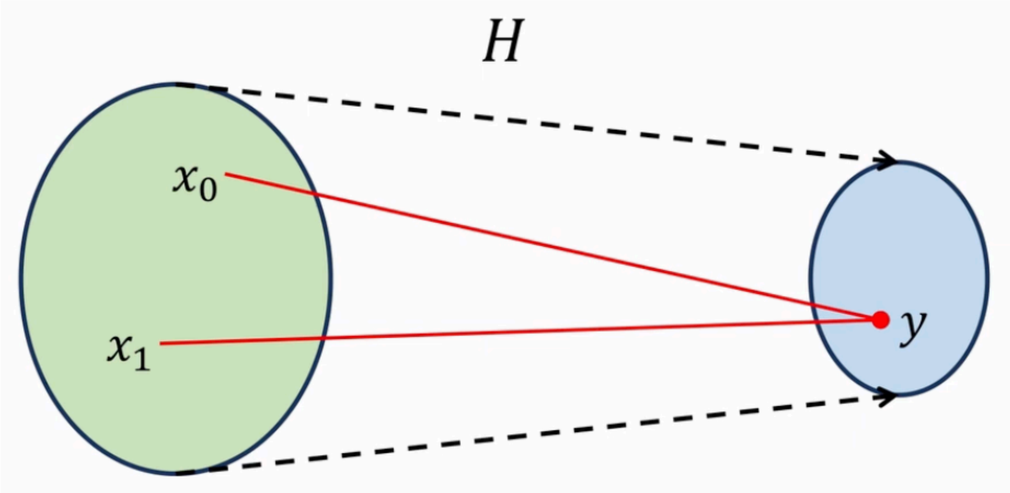
How to Construct a One-Shot Signature

- Post-Quantum Collision Resistance: given hash function H, it is hard to find x_0 and x_1 such that $H(x_0) = H(x_1) = y$



How to Construct a One-Shot Signature

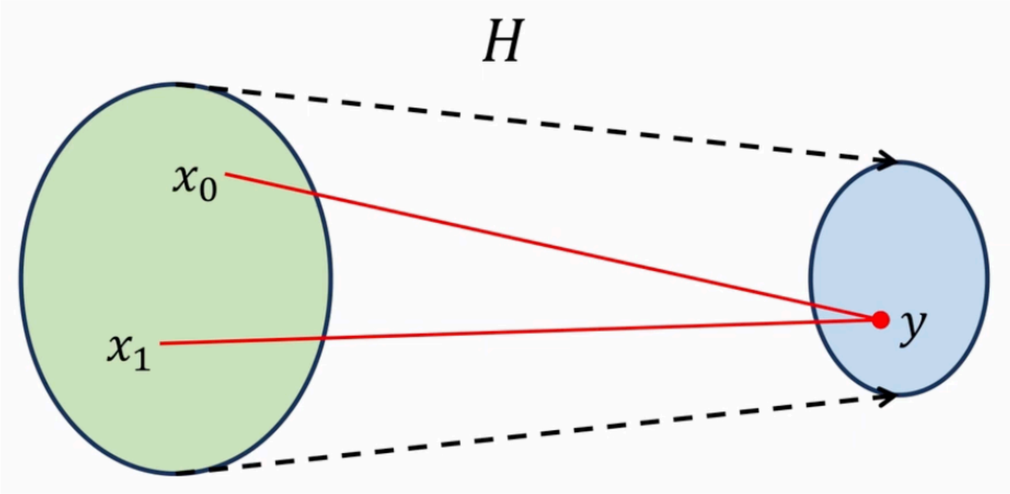
- Post-Quantum Collision Resistance: given hash function H, it is hard to find x_0 and x_1 such that $H(x_0) = H(x_1) = y$



- Classically, this means that after sending y, the adversary is "committed" to a preimage: it cannot on input b choose x_b such that $H(x_b) = y$ but $x_0 \neq x_1$

How to Construct a One-Shot Signature

- Post-Quantum Collision Resistance: given hash function H, it is hard to find x_0 and x_1 such that $H(x_0) = H(x_1) = y$



- Classically, this means that after sending y, the adversary is "committed" to a preimage: it cannot on input b choose x_b such that $H(x_b) = y$ but $x_0 \neq x_1$
- Quantumly, this is not known to be equivalent!

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message

- Basic issue:

$$\sum_{x \in \{0,1\}^n} |x\rangle$$

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message

- Basic issue:

$$\sum_{x\in\{0,1\}^n} |x\rangle \longrightarrow \sum_{x\in\{0,1\}^n} |x\rangle\,|H(x)\rangle$$

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message

- Basic issue:

$$\sum_{x\in\{0,1\}^n} |x\rangle \longrightarrow \sum_{x\in\{0,1\}^n} |x\rangle\,|H(x)\rangle \longrightarrow \sum_{x:\,H(x)=y} |x\rangle, y$$

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message
- Basic issue:

$$\sum_{x\in\{0,1\}^n} |x\rangle \longrightarrow \sum_{x\in\{0,1\}^n} |x\rangle\,|H(x)\rangle \longrightarrow \sum_{x:\,H(x)=y} |x\rangle, y$$

- Adversary commits to y, but it still has the state $|H^{-1}(y)\rangle := \sum_{x:\,H(x)=y} |x\rangle$!

How to Construct a One-Shot Signature

- [ARU14]: Relative to a unitary oracle, there exists post-quantum secure CRHF where an adversary can open to *any* message
- Basic issue:

$$\sum_{x\in\{0,1\}^n} |x\rangle \longrightarrow \sum_{x\in\{0,1\}^n} |x\rangle\,|H(x)\rangle \longrightarrow \sum_{x:\,H(x)=y} |x\rangle, y$$

- Adversary commits to y, but it still has the state $|H^{-1}(y)\rangle := \sum_{x:\,H(x)=y} |x\rangle$!
- In theory, it might be able to transform $|H^{-1}(y)\rangle$ to get preimage of its choosing

How to Construct a One-Shot Signature

- [Unruh16]: H is *collapsing* if

$$|H^{-1}(y)\rangle := \sum_{x:\, H(x)=y} |x\rangle \approx_c \{x \leftarrow \{x : H(x) = y\}\}$$

How to Construct a One-Shot Signature

- [Unruh16]: H is *collapsing* if

$$|H^{-1}(y)\rangle := \sum_{x:\, H(x)=y} |x\rangle \approx_c \{x \leftarrow \{x : H(x) = y\}\}$$

- We now know many constructions of collapsing hash functions

How to Construct a One-Shot Signature

- [Unruh16]: H is *collapsing* if

$$|H^{-1}(y)\rangle := \sum_{x:\, H(x)=y} |x\rangle \approx_c \{x \leftarrow \{x : H(x) = y\}\}$$

- We now know many constructions of collapsing hash functions
- Collapsing $\Longrightarrow$ collision resistance, but not necessarily the other way around

How to Construct a One-Shot Signature

- [Unruh16]: H is *collapsing* if

$$|H^{-1}(y)\rangle := \sum_{x:\, H(x)=y} |x\rangle \approx_c \{x \leftarrow \{x : H(x) = y\}\}$$

- We now know many constructions of collapsing hash functions
- Collapsing $\Longrightarrow$ collision resistance, but not necessarily the other way around
- [Zha17, AGKZ20, DS22]: non-collapsing CRHF $\Longrightarrow$ one-shot signature!

Non-Collapsing CRHFs

- How do we construct post-quantum secure non-collapsing but collision-resistant hash functions?

Non-Collapsing CRHFs

- How do we construct post-quantum secure non-collapsing but collision-resistant hash functions?

- Attempt 0: consider the compressing function $H_\pi(x) = \pi(x)_{[1:r]}$ obtained by truncating the image of a random permutation π to the first $r < n$ bits.

Non-Collapsing CRHFs

- How do we construct post-quantum secure non-collapsing but collision-resistant hash functions?
- Attempt 0: consider the compressing function $H_\pi(x) = \pi(x)_{[1:r]}$ obtained by truncating the image of a random permutation π to the first $r < n$ bits.
- **Problem:** H_π is not even classically one-way when given access to π^{-1}!

Non-Collapsing CRHFs

- How do we construct post-quantum secure non-collapsing but collision-resistant hash functions?
- Attempt 0: consider the compressing function $H_\pi(x) = \pi(x)_{[1:r]}$ obtained by truncating the image of a random permutation π to the first $r < n$ bits.
- **Problem:** H_π is not even classically one-way when given access to π^{-1}!
- Given $y \in \{0,1\}^r$, an adversary can choose any $z \in \{0,1\}^{n-r}$ and compute a preimage $x = \pi^{-1}(y \| z)$.

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π
- Let $H_\pi(x) = \pi(x||0^s)_{[1:r]}$, where $n > r + s$ (similar to the single round sponge construction).

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π
- Let $H_\pi(x) = \pi(x||0^s)_{[1:r]}$, where $n > r + s$ (similar to the single round sponge construction).
- Prior attack does not apply any more, since it is difficult to find suffixes z such that $\pi^{-1}(y||z)$ ends with 0^s.

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π
- Let $H_\pi(x) = \pi(x||0^s)_{[1:r]}$, where $n > r + s$ (similar to the single round sponge construction).
- Prior attack does not apply any more, since it is difficult to find suffixes z such that $\pi^{-1}(y||z)$ ends with 0^s.
- Collision resistance against classical adversaries is straightforward to show

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π
- Let $H_\pi(x) = \pi(x||0^s)_{[1:r]}$, where $n > r + s$ (similar to the single round sponge construction).
- Prior attack does not apply any more, since it is difficult to find suffixes z such that $\pi^{-1}(y||z)$ ends with 0^s.
- Collision resistance against classical adversaries is straightforward to show
- Turns out this construction is post-quantum collision-resistant [Zha21,Car24]

Non-Collapsing CRHFs

- **Attempt 1:** Restrict the domain to which one applies π
- Let $H_\pi(x) = \pi(x||0^s)_{[1:r]}$, where $n > r + s$ (similar to the single round sponge construction).
- Prior attack does not apply any more, since it is difficult to find suffixes z such that $\pi^{-1}(y||z)$ ends with 0^s.
- Collision resistance against classical adversaries is straightforward to show
- Turns out this construction is post-quantum collision-resistant [Zha21,Car24]
- **Bad news:** this construction is actually collapsing!

Non-Collapsing CRHFs

- **Attempt 2:** Restrict the *range* of the hash function
- Consider expanding functions $G : \{0,1\}^n \to \{0,1\}^m$ and let $H(x) = G(x)_{[1:r]}$

Non-Collapsing CRHFs

- **Attempt 2:** Restrict the *range* of the hash function
- Consider expanding functions $G : \{0,1\}^n \to \{0,1\}^m$ and let $H(x) = G(x)_{[1:r]}$
- **Gated permutation:** instead of directly exposing access to π and π^{-1}, apply an expanding function to the output of a random permutation π. Letting $\pi(x) = y \| z$, define $G_{\pi,F}(x) := y \| F(y,z)$ where $F(y, \cdot\ )$ is an injective function all y. This also defines an inverse function $G^{-1}_{\pi,F}$.

Non-Collapsing CRHFs

- **Attempt 2:** Restrict the *range* of the hash function
- Consider expanding functions $G : \{0,1\}^n \to \{0,1\}^m$ and let $H(x) = G(x)_{[1:r]}$
- **Gated permutation:** instead of directly exposing access to π and π^{-1}, apply an expanding function to the output of a random permutation π. Letting $\pi(x) = y \| z$, define $G_{\pi,F}(x) := y \| F(y,z)$ where $F(y, \cdot)$ is an injective function all y. This also defines an inverse function $G^{-1}_{\pi,F}$.
- We show that when each $F(y, \cdot)$ is independently drawn from a family $\mathscr{F}$ with mild statistical properties (e.g. bounded preimages/pairwise independent), then H is one-way/collision-resistant when given access to $(G_{\pi,F}, G^{-1}_{\pi,F})$

Proof Sketch

- For simplicity, we focus on the case of preimage finding, where the goal is to find a preimage of some fixed y

Proof Sketch

- For simplicity, we focus on the case of preimage finding, where the goal is to find a preimage of some fixed y
- Let's think about the proof in the *classical* setting, which begins by simulating $G_{\pi,F}$ on-the-fly via *lazy sampling*

Proof Sketch

- For simplicity, we focus on the case of preimage finding, where the goal is to find a preimage of some fixed y
- Let's think about the proof in the *classical* setting, which begins by simulating $G_{\pi,F}$ on-the-fly via *lazy sampling*
- To simulate access to π and F, we will randomly sample input-output pairs whenever a query is made whilst ensuring consistency

Proof Sketch

- For simplicity, we focus on the case of preimage finding, where the goal is to find a preimage of some fixed y
- Let's think about the proof in the *classical* setting, which begins by simulating $G_{\pi,F}$ on-the-fly via *lazy sampling*
- To simulate access to π and F, we will randomly sample input-output pairs whenever a query is made whilst ensuring consistency
- The adversary starts with no knowledge, corresponding to an empty database $D = \emptyset$

Proof Sketch

- For simplicity, we focus on the case of preimage finding, where the goal is to find a preimage of some fixed y
- Let's think about the proof in the *classical* setting, which begins by simulating $G_{\pi,F}$ on-the-fly via *lazy sampling*
- To simulate access to π and F, we will randomly sample input-output pairs whenever a query is made whilst ensuring consistency
- The adversary starts with no knowledge, corresponding to an empty database $D = \emptyset$
- Whenever a new query is made, we can sample a new input-output pair, provide these values, and update D

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \mathrm{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \text{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.
  - Thus, given q forward queries, we expect one of them to match y with probability roughly $O(q/2^r)$.

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \text{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.
  - Thus, given q forward queries, we expect one of them to match y with probability roughly $O(q/2^r)$.
- *Inverse queries*: if the query is of the form (y', w) where $y' \neq y$, then we can simply sample a random function $F(y', \cdot)$ and check if w is in the range before updating accordingly

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \mathsf{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.
  - Thus, given q forward queries, we expect one of them to match y with probability roughly $O(q/2^r)$.
- *Inverse queries*: if the query is of the form (y', w) where $y' \neq y$, then we can simply sample a random function $F(y', \cdot)$ and check if w is in the range before updating accordingly
- To handle queries of the form (y, w), we note that if D does not contain y, the adversary must guess if $w \in \mathsf{im}(F(y, \cdot))$, which occurs with probability at most $p_1 = \max_v \Pr_{F \sim \mathscr{F}}[v \in \mathsf{im}(F(y, \cdot))]$.

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \mathsf{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.
  - Thus, given q forward queries, we expect one of them to match y with probability roughly $O(q/2^r)$.
- *Inverse queries*: if the query is of the form (y', w) where $y' \neq y$, then we can simply sample a random function $F(y', \cdot)$ and check if w is in the range before updating accordingly
- To handle queries of the form (y, w), we note that if D does not contain y, the adversary must guess if $w \in \mathsf{im}(F(y, \cdot))$, which occurs with probability at most $p_1 = \max_v \Pr_{F \sim \mathcal{F}}[v \in \mathsf{im}(F(y, \cdot))]$.
- Thus, q inverse queries should succeed in updating D to contain y with probability $O(q \cdot p_1)$

Proof Sketch

- Forward queries: whenever an adversary makes a query $x \notin \mathsf{Dom}(D)$ to $G_{\pi,F}$, $\pi(x)_{[1:r]}$ will take on a random* value in $\{0,1\}^r$.
  - Thus, given q forward queries, we expect one of them to match y with probability roughly $O(q/2^r)$.
- *Inverse queries*: if the query is of the form (y', w) where $y' \neq y$, then we can simply sample a random function $F(y', \cdot)$ and check if w is in the range before updating accordingly
- To handle queries of the form (y, w), we note that if D does not contain y, the adversary must guess if $w \in \mathsf{im}(F(y, \cdot))$, which occurs with probability at most $p_1 = \max_v \Pr_{F \sim \mathcal{F}}[v \in \mathsf{im}(F(y, \cdot))]$.
- Thus, q inverse queries should succeed in updating D to contain y with probability $O(q \cdot p_1)$
- Combining our two bounds gives an upper bound of roughly $O(q \cdot (2^{-r} + p_1))$

Proof Sketch

- How do we quantize this?

Proof Sketch

- How do we quantize this?
- We can analyze random functions using the *compressed oracle* framework [Zha19]

Proof Sketch

- How do we quantize this?
- We can analyze random functions using the *compressed oracle* framework [Zha19]
  - The work of [Car25], building on [ACMT25], shows how to analyze random permutations via the *compressed permutation oracle*

Proof Sketch

- How do we quantize this?
- We can analyze random functions using the *compressed oracle* framework [Zha19]
  - The work of [Car25], building on [ACMT25], shows how to analyze random permutations via the *compressed permutation oracle*
- Issue: $G_{\pi,F}$ is the composition of a random permutation with a function (which is not necessarily fully random)

Proof Sketch

- How do we quantize this?
- We can analyze random functions using the *compressed oracle* framework [Zha19]
  - The work of [Car25], building on [ACMT25], shows how to analyze random permutations via the *compressed permutation oracle*
- Issue: $G_{\pi,F}$ is the composition of a random permutation with a function (which is not necessarily fully random)
  - Given output $(x, y||v)$ of $G_{\pi,F}$, there are multiple values of z such that $\pi(x) = y||z$ is consistent with seeing v

Proof Sketch

- How do we quantize this?
- We can analyze random functions using the *compressed oracle* framework [Zha19]
  - The work of [Car25], building on [ACMT25], shows how to analyze random permutations via the *compressed permutation oracle*
- Issue: $G_{\pi,F}$ is the composition of a random permutation with a function (which is not necessarily fully random)
  - Given output $(x, y||v)$ of $G_{\pi,F}$, there are multiple values of z such that $\pi(x) = y||z$ is consistent with seeing v
- Solution: Purify random coins r in F and coherently track consistent pairs (r, z)

Recap

- If we instantiate $\mathscr{F}$ appropriately, it is simple to construct a collapsing distinguisher [SZ25, HSVZ26]

Recap

- If we instantiate $\mathscr{F}$ appropriately, it is simple to construct a collapsing distinguisher [SZ25, HSVZ26]

- Applications: optimal/tight constructions of non-collapsing CRHFs, one-shot signatures, and quantum fire schemes in the classical oracle model

Recap

- If we instantiate $\mathscr{F}$ appropriately, it is simple to construct a collapsing distinguisher [SZ25, HSVZ26]
- Applications: optimal/tight constructions of non-collapsing CRHFs, one-shot signatures, and quantum fire schemes in the classical oracle model
- Also: more general version of the fundamental lemma which relates an algorithm's output to the values recorded in the compressed database